

## 离散数学作业 (六)

1、设 $G$ 是具有 $n$ 个顶点和 $m$ 条边的二分图, 证明 $m \leq n^2/4$ .

设 $G$ 分为 $G_1, G_2$ 两部分,  $\forall v, u \in G_1$  或  $v, u \in G_2$

$(v, u) \notin E$ ,  $G_1, G_2$ 分别有 $n_1, n_2$ 个点

$$n_1 + n_2 = n$$

$$\text{则 } m \leq n_1 \cdot n_2 = n_1(n - n_1) \leq \frac{n^2}{4}$$

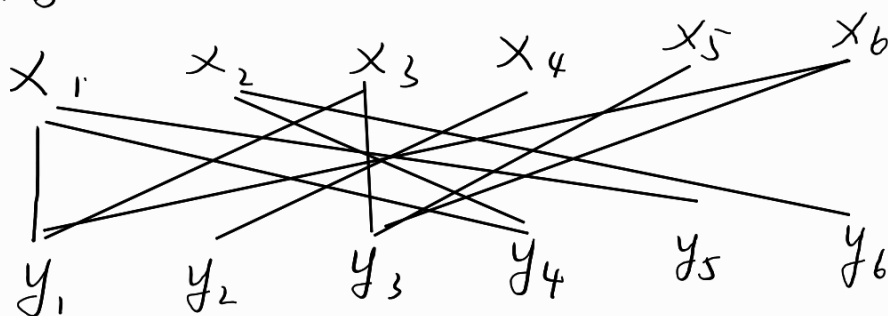
2、证明顶点个数为奇数的二分图不是哈密顿图.

设 $G$ 分为两部分 $G_1, G_2$

若 $G$ 是哈密顿图, 则必存在一条回路从 $G_1$ 中某点 $v$ 出发  
遍历所有点后回到 $v$ , 必在 $G_1, G_2$ 中往返若干次后回到 $G_1$ ,  
而顶点个数为奇数, 则遍历点后将停在 $G_2$ 矛盾

3、有 $x_1, x_2, x_3, x_4, x_5, x_6$ 六个工人, 要分配他们做 $y_1, y_2, y_3, y_4, y_5, y_6$ 六项工作. 但并不是每个工人都能做任何一项工作.  $x_1$ 只能做 $y_1, y_4$ 和 $y_5$ ;  $x_2$ 只能做 $y_4$ 和 $y_6$ ;  $x_3$ 只能做 $y_1$ 和 $y_3$ ;  $x_4$ 只能做 $y_2$ ;  $x_5$ 只能做 $y_3$ ;  $x_6$ 只能做 $y_1$ 和 $y_3$ . 问能否通过适当的安排, 使每项工作都由能做的工人做? 如果做不到, 怎样安排才能使尽可能多的工作有人做?

抽象为二分图:



共有六项工作

若存在完备匹配

$$\text{则 } y_1 = x_4$$

$$y_5 = x_2$$

$y_6 = x_1$ , 而此时  $y_4$  无人匹配, 矛盾, 故最大边数为 5

另找一个匹配

$$y_2 = x_4$$

$$y_5 = x_2$$

$$y_6 = x_1$$

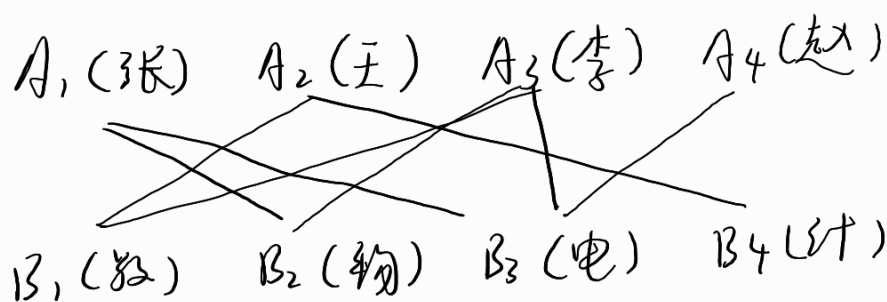
$$y_1 = x_3$$

$$y_3 = x_6$$

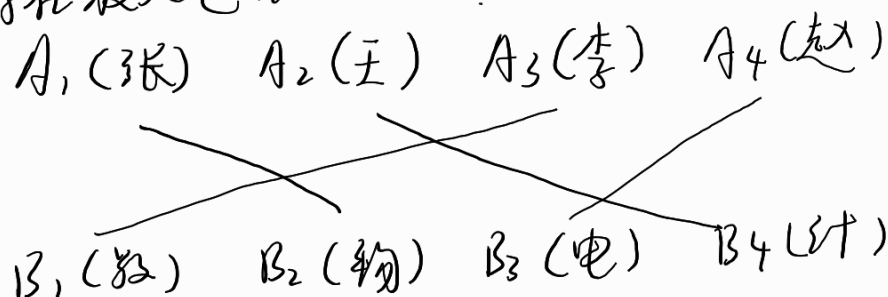
已达到最大

4、有姓张、王、李和赵的四名教师, 要分配他们教数学、物理、电工和计算机语言四门课程. 张懂物理和电工, 王懂数学和计算机语言, 李懂数学、物理和电工, 赵只懂电工. 应如何分派使他们能适得其所?

抽象为二分图.



存在最大匹配如下:



5、一棵树有  $n_2$  个顶点次数为 2,  $n_3$  个顶点次数为 3,  $\dots$ ,  $n_k$  个顶点次

数为 $k$ , 问这棵树有几片树叶?

设有 $n_i$ 个顶点度数为 $i$

总边数  $m = \sum n_i - 1$

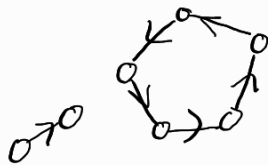
$$2m = \sum i n_i$$

$$\text{得 } n_1 = 2 + \sum_{i=3}^k (i-2) n_i$$

若根节点度数为 $1$ , 则有  $1 + \sum_{i=3}^k (i-2) n_i$  片

否则有  $2 + \sum_{i=3}^k (i-2) n_i$

6、试举例说明: 在一有向图中, 如果仅有一个顶点的引入次数为 $0$ , 而其他顶点的引入次数都为 $1$ , 这样的有向图不一定是树。



7、如何根据有向图的邻接矩阵确定一个有向树是否是根树? 如果它是根树, 如何从它的邻接矩阵确定树根和树叶?

1. 主对角线上全为 $0$
2. 有一列全为 $0$ , 此为根
3. 若一行全为 $0$ , 此为树叶

8、证明完全二元树有奇数个顶点。

设有 $n$ 层 (包括根节点)

共有  $2^n - 1$  个顶点, 显然是奇数个

9、图 1 给出了一有序树，试求其对应的位置二元树.

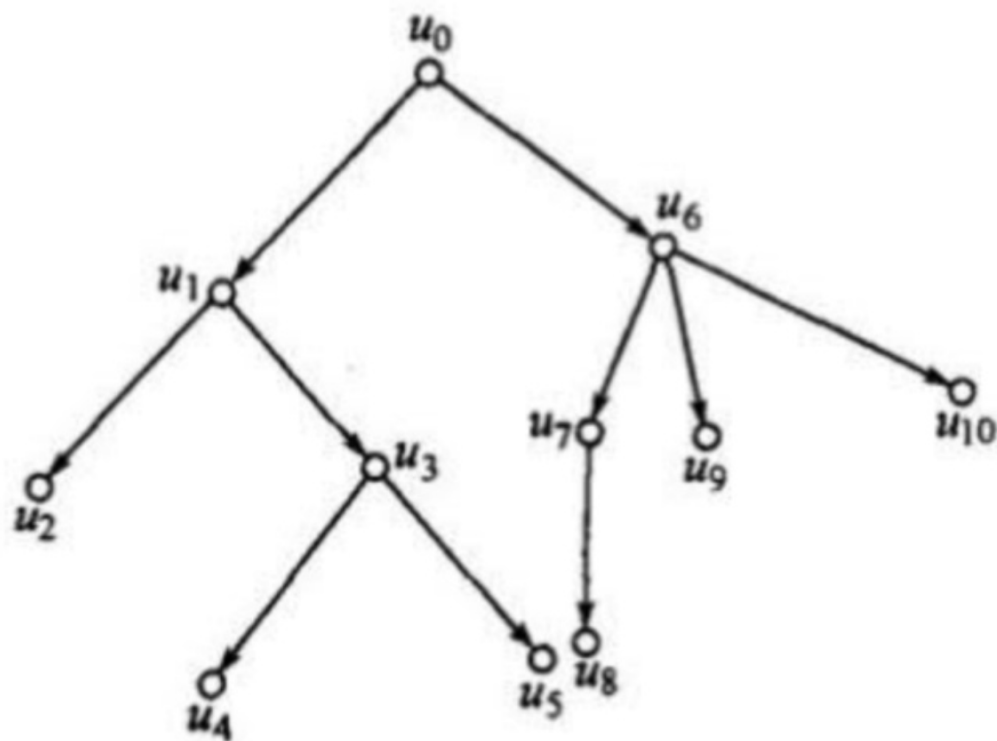
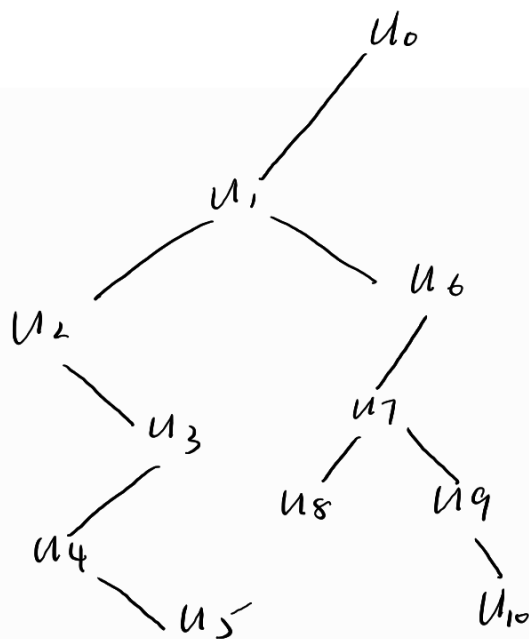


图 1



10、设  $T_1$  和  $T_2$  是连通无向图  $G$  的两个生成树,  $a$  是在  $T_1$  中但是不在  $T_2$  中的一条边. 证明: 存在边  $b$ , 它在  $T_2$  中但不在  $T_1$  中, 使得  $T_1 - a + b$  和  $T_2 - b + a$  都是  $G$  的生成树.

$a$  在  $T_1$  中, 通过  $a$  有一个割集  $D$

$a$  不在  $T_2$  中,  $\therefore$  有包含  $a$  的回路  $C$

而D与C有公共边a, 则必有另一边b  
b在T<sub>2</sub>中

∴ T<sub>1</sub> - a + b 仍是G生成树

同理 T<sub>2</sub> - b + a 仍是G生成树

11、下表为一些文件中各字符出现频率统计表. 采用霍夫曼编码对下列字符进行编码, 并给出字符序列“BEI HANG”对应的编码.

| 符号 | 频率   | 符号 | 频率   |
|----|------|----|------|
| A  | 7.7% | N  | 6.7% |
| B  | 1.7% | O  | 6.7% |
| C  | 3.2% | P  | 2%   |
| D  | 4.2% | Q  | 0.5% |
| E  | 12%  | R  | 5.9% |
| F  | 2.4% | S  | 6.7% |
| G  | 1.7% | T  | 8.5% |
| H  | 5%   | U  | 3.7% |
| I  | 7.6% | V  | 1.2% |
| J  | 0.4% | W  | 2.2% |
| K  | 0.7% | X  | 0.4% |
| L  | 4.2% | Y  | 2.2% |
| M  | 2.4% | Z  | 0.2% |

编写python代码→

```
import heapq
from collections import defaultdict

# 哈夫曼树节点类
class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

# 构建哈夫曼树
def build_huffman_tree(frequencies):
    priority_queue = [Node(chr(i + 65), freq) for i, freq in enumerate(frequencies)]
    heapq.heapify(priority_queue)

    while len(priority_queue) > 1:
        left = heapq.heappop(priority_queue)
        right = heapq.heappop(priority_queue)
        merged = Node(None, left.freq + right.freq)
        merged.left = left
        merged.right = right
        heapq.heappush(priority_queue, merged)

    return priority_queue[0]

# 生成哈夫曼编码
def generate_huffman_codes(node, current_code="", codes={}):
    if node:
        if node.char is not None:
            codes[node.char] = current_code
            generate_huffman_codes(node.left, current_code + "0", codes)
            generate_huffman_codes(node.right, current_code + "1", codes)
        return codes

# 示例频率数组 (假设 A-Z 的频率)
frequencies = [7.7, 1.7, 3.2, 4.2, 12, 2.4, 1.7, 5, 7.6, 0.4, 0.7, 4.2, 12, 2.4, 6.7, 6.7, 2, 0.5, 5.9, 6.7, 8.5, 3.7, 1.2, 2.2, 0.4, 2.2, 0.2]

# 构建哈夫曼树
root = build_huffman_tree(frequencies)

# 生成哈夫曼编码
huffman_codes = generate_huffman_codes(root)

# 输出哈夫曼编码
for char, code in huffman_codes.items():
    print(f"{char}: {code}")
```

D: 0000

J: 0001000

Q: 0001001

V: 000101

Y: 00011

W: 00100

F: 00101

H: 0011

M: 01000

Z: 01001000

X: 01001001

K: 0100101

G: 010011

↪ output

对应 BEI HANG

↓

11100001111000001111011010010011

R: 0101  
E: 011  
O: 1000  
S: 1001  
N: 1010  
C: 10110  
U: 10111  
I: 1100  
A: 1101  
B: 111000  
P: 111001  
L: 11101  
T: 1111